

CleanHome

Manual Técnico del Sistema

Guía de implementación, configuración, ejecución, integración y mantenimiento

Campo	Detalle
Tipo de documento	Manual técnico
Proyecto	CleanHome - Gestión de servicios de limpieza a domicilio
Repositorio base	https://github.com/K3v94K/CleanHome
Fecha de elaboración	23 de mayo de 2026
Versión del documento	1.0
Audiencia objetivo	Docentes, evaluadores técnicos y desarrolladores del proyecto

Elaborado a partir del repositorio público CleanHome y del documento técnico de referencia proporcionado.

Control documental

Versión	Fecha	Descripción del cambio	Responsable
1.0	23/05/2026	Primera versión consolidada del manual técnico en formato Word.	Equipo del proyecto CleanHome
Nota de uso: Este documento describe el estado técnico del repositorio revisado al momento de su elaboración. Si el código cambia, se recomienda actualizar las secciones de endpoints, dependencias, scripts de base de datos y flujo de ejecución.			

Contenido

Control documental	2
1. Objetivo del manual	5
2. Alcance técnico	5
3. Descripción general del sistema	5
4. Arquitectura general	6
5. Tecnologías utilizadas	6
6. Requisitos previos	7
7. Estructura del repositorio	7
8. Backend REST	8
8.1 Estructura del backend	8
8.2 Dependencias del backend	8
8.3 Archivos principales	9
8.4 Middlewares de seguridad	9
9. Configuración del backend	9
9.1 Ejecución del backend	10
10. Base de datos MySQL	10
10.1 Creación desde cero	10
10.2 Migración para bases existentes	10
10.3 Tablas principales	11
10.4 Relaciones principales	11
10.5 Datos semilla	11
11. API REST y endpoints	11
11.1 Autenticación	12
11.2 Servicios	13
11.3 Solicitudes de cliente	13
11.4 Administración	13
12. Frontend web administrativo	13
12.1 Funcionalidades del panel	14
12.2 Ejecución del frontend web	14
13. Aplicación Android	14
13.1 Configuración de API	14
13.2 Permisos Android	15
13.3 Actividades principales	15

13.4 Flujo de cliente Android	15
13.5 Flujo de administrador Android	15
14. SQLite local en Android	17
14.1 Relación con MySQL	17
15. Flujo de levantamiento completo	17
16. Pruebas de integración recomendadas	19
17. Seguridad y buenas prácticas	19
18. Solución de problemas	19
19. Mantenimiento	20
20. Glosario	20
21. Conclusiones	21

1. Objetivo del manual

El presente manual técnico tiene como objetivo documentar la estructura, configuración, ejecución y mantenimiento del sistema CleanHome, de forma que un desarrollador pueda levantar el proyecto desde un ambiente limpio, comprender sus módulos principales y validar la integración entre backend, frontend web, aplicación Android y bases de datos.

El documento está orientado a un contexto universitario, por lo que prioriza claridad, trazabilidad técnica y pasos reproducibles. También sirve como apoyo para evaluación académica, mantenimiento del código y transferencia de conocimiento entre integrantes del equipo.

2. Alcance técnico

El manual cubre los siguientes componentes del repositorio CleanHome:

- Backend REST desarrollado con Node.js, Express y MySQL.
- Base de datos central MySQL creada mediante el script `cleanhome-backend/database/cleanhome.sql`.
- Panel web administrativo servido desde `cleanhome-admin-frontend`.
- Aplicación Android desarrollada en Kotlin con vistas XML.
- Consumo de API desde Android mediante Retrofit y Gson.
- Persistencia local Android mediante SQLiteOpenHelper y SharedPreferences.
- Autenticación mediante JWT y separación de permisos por roles Cliente y Admin.
- Flujos principales de servicios, solicitudes, estados, personal de limpieza y sincronización local.

Límite del alcance: El manual no sustituye una guía de usuario final. Se enfoca en configuración, arquitectura, ejecución, endpoints, base de datos, validación técnica y mantenimiento del proyecto.

3. Descripción general del sistema

CleanHome es un sistema para gestionar servicios de limpieza a domicilio. El proyecto permite que clientes consulten un catálogo de servicios, registren solicitudes y revisen su historial. Además, incluye un módulo administrativo para revisar solicitudes, cambiar estados, asignar personal y gestionar el catálogo de servicios.

La solución está separada en tres aplicaciones principales: un backend REST que centraliza reglas de negocio y acceso a MySQL; un frontend web administrativo; y una aplicación Android que consume la API y conserva datos locales para sesión, catálogo e historial.

Componente	Responsabilidad principal
Backend REST	Expone endpoints HTTP, valida autenticación, aplica reglas de negocio y accede a MySQL.

Componente	Responsabilidad principal
Base MySQL	Almacena usuarios, roles, servicios, personal de limpieza y solicitudes.
Frontend admin	Permite gestionar solicitudes, personal y servicios desde navegador.
App Android	Permite registro, login, consulta de servicios, creación de solicitudes, historial y funciones básicas de administración.
SQLite Android	Permite cache local de sesión, servicios, solicitudes y soporte para sincronización offline.

4. Arquitectura general

La arquitectura separa presentación, servicios y persistencia. El backend es la única capa que se comunica directamente con MySQL; el panel web y la app Android consumen la API REST.

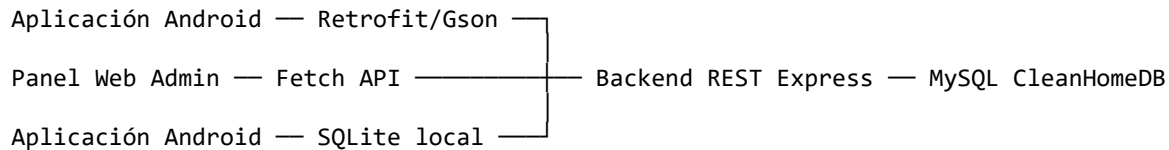


Figura 1. Vista simplificada de arquitectura lógica de CleanHome.

Capa	Elemento	Descripción
Presentación	Aplicación Android	Interfaz móvil para usuarios cliente y administración básica.
Presentación	Panel web administrativo	Interfaz web para administración de solicitudes, personal y servicios.
Servicios	Backend REST Express	API central para autenticación, servicios, solicitudes y operaciones administrativas.
Persistencia central	MySQL	Fuente oficial de datos del sistema.
Persistencia local	SQLite Android	Cache local y soporte para sincronización en el dispositivo.

5. Tecnologías utilizadas

Área	Tecnología	Uso dentro del proyecto
Backend	Node.js	Ejecución del servidor backend.
Backend	Express	Definición de API REST, middlewares y rutas HTTP.
Backend	mysql2/promise	Conexión a MySQL mediante pool de conexiones.
Backend	dotenv	Carga de variables de entorno desde archivo .env.
Backend	bcrypt	Hash y validación de contraseñas.
Backend	jsonwebtoken	Generación y validación de JWT.
Backend	cors	Habilitación de consumo desde frontend web y móvil.
Base de datos	MySQL	Persistencia central del sistema.
Frontend web	HTML, CSS, JavaScript	Panel administrativo ejecutado en navegador.

Área	Tecnología	Uso dentro del proyecto
Frontend web	Node.js HTTP server	Servidor estático para publicar archivos del panel.
Android	Kotlin	Lenguaje principal de la app móvil.
Android	XML Views	Definición de interfaces de usuario Android.
Android	Retrofit + Gson	Consumo de API REST y serialización JSON.
Android	SQLiteOpenHelper	Gestión de base local en el dispositivo.
Android	SharedPreferences	Almacenamiento local de token y datos de sesión.

6. Requisitos previos

Antes de levantar el proyecto, el equipo de desarrollo debe contar con los siguientes elementos instalados y disponibles:

- Git instalado para clonar o descargar el repositorio.
- Node.js y npm instalados.
- MySQL Server instalado y en ejecución.
- Android Studio instalado.
- Emulador Android o dispositivo físico conectado.
- Navegador web moderno para validar el panel administrativo.
- Terminal PowerShell, CMD, Bash o equivalente.
- Puerto 3000 disponible para backend.
- Puerto 5173 disponible para frontend administrativo.

7. Estructura del repositorio

El repositorio CleanHome organiza el código por módulo funcional. La estructura principal identificada es la siguiente:

```
CleanHome/
| - README.md
| - .gitignore
| - cleanhome-backend/
| - cleanhome-admin-frontend/
| - cleanHome-androidApp/
| - documentacion/
| - cdocumentacion/
| - tools/
```

Ruta	Descripción
README.md	Guía general del proyecto, módulos principales, URLs y flujo de validación.
.gitignore	Exclusión de archivos locales o generados, como node_modules, .env, build, .gradle y local.properties.
cleanhome-backend/	API REST, controladores, rutas, middlewares, configuración de MySQL y scripts de base de datos.
cleanhome-admin-frontend/	Panel web administrativo servido como archivos estáticos.

Ruta	Descripción
cleanHome-androidApp/	Aplicación Android para cliente y administración básica.
documentacion/	Documentación técnica generada o de referencia.
cdocumentacion/	Documentación histórica del proyecto.
tools/	Herramientas auxiliares para generación o soporte documental.

8. Backend REST

El backend se ubica en cleanhome-backend y funciona como API REST. Este módulo ya no sirve interfaz web; su responsabilidad es exponer endpoints para autenticación, catálogo de servicios, solicitudes y administración.

8.1 Estructura del backend

```
cleanhome-backend/
|- README.md
|- package.json
|- .env.example
|- database/
|   |- cleanhome.sql
|   |- sqlite_android_cleanhome.sql
|   `-- migrations/
|       `-- 001_sync_support.sql
`-- src/
    |- app.js
    |- server.js
    |- config/
    |   `-- db.js
    |- controllers/
    |   |- auth.controller.js
    |   |- servicios.controller.js
    |   |- solicitudes.controller.js
    |   `-- admin.controller.js
    |- middlewares/
    |   |- auth.middleware.js
    |   `-- role.middleware.js
    `-- routes/
        |- auth.routes.js
        |- servicios.routes.js
        |- solicitudes.routes.js
        `-- admin.routes.js
```

8.2 Dependencias del backend

Dependencia	Uso
express	Framework HTTP para definir rutas y middlewares.
mysql2	Conexión con MySQL mediante API basada en promesas.
dotenv	Lectura de variables de entorno desde .env.
bcrypt	Hash y validación de contraseñas.
jsonwebtoken	Generación y validación de tokens JWT.

Dependencia	Uso
cors	Permite solicitudes desde web y móvil durante desarrollo.
nodemon	Dependencia de desarrollo para reinicio automático del servidor.

8.3 Archivos principales

Archivo	Función técnica
src/server.js	Carga dotenv, obtiene PORT y levanta el servidor Express.
src/app.js	Configura Express, CORS, JSON, rutas del API, fallback 404 y manejador de errores.
src/config/db.js	Configura el pool de conexiones MySQL usando variables de entorno.
routes/auth.routes.js	Define POST /register y POST /login.
routes/servicios.routes.js	Define endpoints públicos del catálogo de servicios.
routes/solicitudes.routes.js	Define endpoints de solicitudes para clientes autenticados.
routes/admin.routes.js	Agrupar endpoints administrativos protegidos con JWT y rol Admin.
middlewares/auth.middleware.js	Valida encabezado Authorization: Bearer <token> y expone req.user.
middlewares/role.middleware.js	Verifica que el usuario autenticado tenga el rol requerido.

8.4 Middlewares de seguridad

El backend utiliza dos middlewares principales para proteger operaciones:

- auth.middleware.js valida que exista un token JWT en el encabezado Authorization, lo verifica con JWT_SECRET y adjunta el payload a req.user.
- role.middleware.js valida que el campo nombre_rol del usuario coincida con el rol requerido. En el caso administrativo, se exige rol Admin.

9. Configuración del backend

La configuración local se realiza por medio de un archivo .env dentro de cleanhome-backend. El repositorio incluye .env.example como referencia.

```
PORT=3000
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=
DB_NAME=CleanHomeDB
JWT_SECRET=cleanhome_secret_key
JWT_EXPIRES_IN=8h
```

Variable	Descripción	Recomendación
PORT	Puerto HTTP del backend.	Usar 3000 en desarrollo.
DB_HOST	Servidor MySQL.	localhost si MySQL corre en la misma máquina.
DB_USER	Usuario de conexión a MySQL.	Usar usuario local con permisos sobre CleanHomeDB.

Variable	Descripción	Recomendación
DB_PASSWORD	Contraseña del usuario MySQL.	No dejar vacío fuera de ambientes locales controlados.
DB_NAME	Nombre de la base de datos.	CleanHomeDB.
JWT_SECRET	Clave usada para firmar tokens JWT.	Cambiar por ambiente y no subir al repositorio.
JWT_EXPIRES_IN	Tiempo de expiración del token.	8h en desarrollo según ejemplo.
Importante: El archivo .env no debe versionarse. Cada desarrollador debe crear su propia configuración local a partir de .env.example.		

9.1 Ejecución del backend

```
cd cleanhome-backend
npm install
npm start
```

URL esperada del backend: <http://localhost:3000>

Validación rápida recomendada: abrir <http://localhost:3000/api/servicios> y confirmar una respuesta JSON con la propiedad servicios.

10. Base de datos MySQL

La base de datos central del sistema se llama CleanHomeDB y se crea a partir del script `cleanhome-backend/database/cleanhome.sql`. MySQL es la fuente oficial de datos del backend.

10.1 Creación desde cero

Para instalaciones limpias, el flujo recomendado es importar directamente el script oficial del proyecto:

```
mysql -u TU_USUARIO -p < cleanhome-backend/database/cleanhome.sql
```

Como alternativa, también se puede crear la base vacía y luego importar el script dentro de CleanHomeDB:

```
mysql -u TU_USUARIO -p
DROP DATABASE IF EXISTS CleanHomeDB;
CREATE DATABASE CleanHomeDB CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
exit
```

```
mysql -u TU_USUARIO -p CleanHomeDB < cleanhome-backend/database/cleanhome.sql
```

10.2 Migración para bases existentes

La migración `cleanhome-backend/database/migrations/001_sync_support.sql` se debe usar solo cuando exista una base anterior que se desea actualizar sin borrar información. En instalaciones desde cero no debe ejecutarse como primer paso.

10.3 Tablas principales

Tabla	Propósito	Campos principales
roles	Define roles del sistema.	id_rol, nombre_rol, descripcion
usuarios	Almacena clientes y administradores.	id_usuario, nombre, correo, telefono, direccion, password_hash, id_rol, fecha_creacion, updated_at
servicios	Catálogo de servicios ofertados.	id_servicio, nombre, descripcion, duracion_estimada, precio, activo, fecha_creacion, updated_at
personal_limpieza	Personal disponible para asignaciones.	id_personal, nombre, telefono, disponible, fecha_creacion, updated_at
solicitudes	Solicitudes de servicios creadas por clientes.	id_solicitud, id_usuario, id_servicio, id_personal, fecha_servicio, hora_servicio, direccion_atencion, estado, client_temp_id

10.4 Relaciones principales

Relación	Descripción
usuarios.id_rol -> roles.id_rol	Cada usuario pertenece a un rol.
solicitudes.id_usuario -> usuarios.id_usuario	Cada solicitud pertenece a un usuario cliente.
solicitudes.id_servicio -> servicios.id_servicio	Cada solicitud corresponde a un servicio solicitado.
solicitudes.id_personal -> personal_limpieza.id_personal	Una solicitud puede tener personal asignado.
UNIQUE(id_usuario, client_temp_id)	Evita duplicar solicitudes sincronizadas desde Android.

10.5 Datos semilla

El script cleanhome.sql incluye datos iniciales para facilitar las pruebas:

- Roles base: Admin y Cliente.
- Servicios iniciales: Limpieza General, Limpieza Profunda y Limpieza de Ventanas.
- Personal de limpieza inicial: Ana Silva, Carlos Ruiz y Fernanda López.
- Usuario administrador por defecto: admin@cleanhome.com con contraseña Admin123!

Recomendación: Las credenciales demo deben cambiarse antes de cualquier despliegue real o presentación pública fuera del entorno académico controlado.

11. API REST y endpoints

La API REST expone rutas públicas, rutas para cliente autenticado y rutas administrativas protegidas por JWT y rol Admin.

Método	Endpoint	Auth	Rol	Uso
POST	/api/auth/register	No	Público	Registrar cliente.
POST	/api/auth/login	No	Público	Iniciar sesión y obtener JWT.

Método	Endpoint	Auth	Rol	Uso
GET	/api/servicios	No	Público	Listar servicios activos.
GET	/api/servicios/:id	No	Público	Consultar detalle de servicio activo.
POST	/api/solicitudes	Sí	Cliente	Crear solicitud de servicio.
GET	/api/solicitudes/mis-solicitudes	Sí	Cliente	Consultar historial propio.
GET	/api/solicitudes/mis-solicitudes?updated_since=fecha	Sí	Cliente	Sincronización incremental.
POST	/api/solicitudes/sync	Sí	Cliente	Sincronizar solicitudes creadas offline.
GET	/api/solicitudes/:id	Sí	Cliente	Consultar solicitud propia por id.
GET	/api/admin/solicitudes	Sí	Admin	Listar todas las solicitudes.
GET	/api/admin/personal	Sí	Admin	Listar personal de limpieza.
PATCH	/api/admin/solicitudes/:id/estado	Sí	Admin	Cambiar estado de solicitud.
PATCH	/api/admin/solicitudes/:id/asignar-personal	Sí	Admin	Asignar personal a solicitud.
GET	/api/admin/servicios	Sí	Admin	Listar servicios activos e inactivos.
POST	/api/admin/servicios	Sí	Admin	Crear servicio.
PUT	/api/admin/servicios/:id	Sí	Admin	Editar servicio.
DELETE	/api/admin/servicios/:id	Sí	Admin	Desactivar servicio lógicamente.

11.1 Autenticación

Registro de cliente:

```
POST /api/auth/register
{
  "nombre": "Cliente Demo",
  "correo": "cliente@demo.com",
  "telefono": "70000000",
  "direccion": "San Salvador",
  "password": "ClaveSegura123"
}
```

Inicio de sesión:

```
POST /api/auth/login
{
  "correo": "cliente@demo.com",
  "password": "ClaveSegura123"
}
```

La respuesta exitosa de login incluye un token JWT y datos básicos del usuario, incluyendo id_usuario, nombre, correo, id_rol y nombre_rol.

11.2 Servicios

El catálogo público devuelve servicios activos. Para sincronización se puede usar `updated_since`, lo que permite recuperar servicios modificados desde una fecha determinada.

```
GET /api/servicios
GET /api/servicios?updated_since=2026-05-23T00:00:00
GET /api/servicios/:id
```

11.3 Solicitudes de cliente

La creación de solicitudes requiere token JWT del cliente.

```
POST /api/solicitudes
Authorization: Bearer <token>
{
  "id_servicio": 1,
  "fecha_servicio": "2026-05-25",
  "hora_servicio": "09:00",
  "direccion_atencion": "San Salvador",
  "client_temp_id": "uuid-generado-por-android"
}
```

El campo `client_temp_id` permite identificar solicitudes creadas desde Android y evitar duplicados durante reintentos de sincronización.

11.4 Administración

Los endpoints administrativos requieren token JWT válido y rol Admin. Los estados permitidos para solicitudes son: Pendiente, Confirmada, En proceso, Completada y Cancelada.

```
PATCH /api/admin/solicitudes/:id/estado
Authorization: Bearer <token_admin>
{
  "estado": "Confirmada"
}

PATCH /api/admin/solicitudes/:id/asignar-personal
Authorization: Bearer <token_admin>
{
  "id_personal": 1
}
```

12. Frontend web administrativo

El módulo `cleanhome-admin-frontend` implementa un panel administrativo separado del backend. Se sirve como frontend estático mediante un servidor Node.js simple definido en `server.js`.

Elemento	Detalle
Ruta	<code>cleanhome-admin-frontend/</code>
Comando de ejecución	<code>npm start</code>
URL esperada	<code>http://localhost:5173</code>

Elemento	Detalle
API Base recomendada	http://localhost:3000/api
Persistencia local del panel	localStorage para token admin y URL base del API.

12.1 Funcionalidades del panel

- Login administrativo con JWT.
- Listado de solicitudes registradas.
- Cambio de estado de solicitudes.
- Asignación de personal de limpieza.
- Listado de servicios activos e inactivos.
- Creación, edición y desactivación lógica de servicios.
- Reflejo de servicios creados o editados en el catálogo consumido por Android.

12.2 Ejecución del frontend web

```
cd cleanhome-admin-frontend
npm start
```

Luego abrir en navegador: http://localhost:5173

13. Aplicación Android

La aplicación Android se ubica en cleanHome-androidApp. Está desarrollada en Kotlin, utiliza vistas XML y consume la API REST mediante Retrofit con conversión JSON por Gson.

Elemento	Detalle
Ruta	cleanHome-androidApp/
IDE recomendado	Android Studio
Lenguaje	Kotlin
Interfaz	Android XML Views
Consumo API	Retrofit + Gson
Persistencia local	SQLiteOpenHelper y SharedPreferences
Base URL emulador	http://10.0.2.2:3000/api/
Base URL dispositivo físico	http://IP_DE_TU_PC:3000/api/

13.1 Configuración de API

La URL base se centraliza en ApiConfig.kt:

```
object ApiConfig {
    // Emulador Android -> PC local.
    // Para celular fisico usa http://IP_DE_TU_PC:3000/api/
    const val BASE_URL = "http://10.0.2.2:3000/api/"
}
```

Precaución: No se deben subir IPs personales al repositorio. En pruebas con celular físico, la IP de la PC debe modificarse solo localmente.

13.2 Permisos Android

El AndroidManifest.xml incluye permisos de red y permite tráfico HTTP local para desarrollo:

- INTERNET
- ACCESS_NETWORK_STATE
- android:usesCleartextTraffic="true"

13.3 Actividades principales

Actividad	Responsabilidad
MainActivity	Pantalla inicial y flujo de login.
RegistroActivity	Registro de cliente.
CatalogoActivity	Consulta y presentación del catálogo de servicios.
AgendarActivity	Agenda de servicio con fecha, hora y dirección.
HistorialActivity	Historial de solicitudes del usuario.
PerfilActivity	Datos de perfil y cierre de sesión.
AdminActivity	Funciones básicas administrativas desde Android.

13.4 Flujo de cliente Android

1. Registro de cliente contra POST /api/auth/register.
2. Login contra POST /api/auth/login.
3. Guardado del token JWT y datos de usuario en SessionManager.
4. Carga del catálogo desde GET /api/servicios.
5. Cache local de servicios en SQLite.
6. Selección de servicio y agenda mediante selectores de fecha y hora.
7. Creación de solicitud contra POST /api/solicitudes.
8. Consulta del historial desde GET /api/solicitudes/mis-solicitudes.
9. Cierre de sesión desde la vista de perfil.

13.5 Flujo de administrador Android

Cuando el usuario autenticado tiene rol Admin, la app permite ingresar a AdminActivity. Las funciones disponibles son:

- Ver solicitudes.
- Cambiar estado de solicitud.

- Ver servicios.
- Crear servicio.
- Editar servicio.
- Desactivar servicio.

La asignación de personal de limpieza se mantiene disponible en el panel web administrative.

14. SQLite local en Android

La base local Android se gestiona mediante CleanHomeDbHelper.kt. Su propósito es guardar datos de sesión, servicios y solicitudes en el dispositivo, además de preparar soporte para sincronización offline.

Tabla local	Uso
usuario_sesion	Datos del usuario autenticado y token.
servicios	Cache local del catálogo de servicios.
solicitudes	Historial local de solicitudes.
solicitudes_pendientes_sync	Cola de solicitudes creadas sin conexión o pendientes de sincronizar.
sync_metadata	Registro de fechas de última sincronización por módulo, definido en el script de referencia SQLite.

14.1 Relación con MySQL

El script `sqlite_android_cleanhome.sql` es una referencia documental para SQLite local. No debe importarse en MySQL. MySQL sigue siendo la base central y oficial; SQLite se usa dentro del dispositivo Android para cache y soporte local.

15. Flujo de levantamiento completo

El siguiente procedimiento permite levantar CleanHome desde cero:

1. Clonar o descargar el repositorio CleanHome.
2. Iniciar MySQL Server.
3. Crear CleanHomeDB desde cero o importar directamente `cleanhome.sql`.
4. Crear `cleanhome-backend/.env` tomando como base `.env.example`.
5. Configurar `DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `JWT_SECRET` y `JWT_EXPIRES_IN`.
6. Ejecutar `npm install` en `cleanhome-backend`.
7. Ejecutar `npm start` en `cleanhome-backend`.
8. Validar `http://localhost:3000/api/servicios`.
9. Ejecutar `npm start` en `cleanhome-admin-frontend`.
10. Abrir `http://localhost:5173`.
11. Iniciar sesión con usuario admin.
12. Crear o editar un servicio desde el panel web.
13. Abrir `cleanHome-androidApp` desde Android Studio.
14. Configurar `ApiConfig.kt` según emulador o dispositivo físico.

15. Ejecutar la app Android.
16. Registrar o iniciar sesión como cliente.
17. Confirmar que el catálogo Android muestra los servicios.
18. Agendar una solicitud.
19. Verificar la solicitud en historial Android y en el panel web/admin.
20. Cambiar estado y confirmar que se refleja en consultas posteriores.

16. Pruebas de integración recomendadas

ID	Prueba	Resultado esperado
PI-01	Validar GET http://localhost:3000/api/servicios .	Backend responde JSON con propiedad servicios.
PI-02	Login admin desde frontend web.	Se obtiene token y se muestra dashboard administrativo.
PI-03	Crear servicio desde panel web.	Servicio se registra en MySQL y aparece en listado admin.
PI-04	Consultar catálogo en Android.	Servicio creado desde web aparece en app Android.
PI-05	Registrar cliente desde Android.	Cliente queda creado y puede iniciar sesión.
PI-06	Crear solicitud desde Android.	Solicitud se registra y aparece en historial del cliente.
PI-07	Consultar solicitudes desde web admin.	La solicitud creada en Android aparece para administración.
PI-08	Cambiar estado desde web o Android admin.	La solicitud queda actualizada con el nuevo estado.
PI-09	Cerrar sesión en Android.	Se limpia la sesión local y no se accede sin nuevo login.
PI-10	Probar token vencido o inválido.	Backend responde 401 y frontend/app solicita nueva sesión.

17. Seguridad y buenas prácticas

- No subir archivos .env ni local.properties al repositorio.
- No dejar JWT_SECRET con valor demo fuera de ambientes académicos o locales.
- No subir IPs personales utilizadas para pruebas con dispositivo físico.
- Usar contraseñas con hash mediante bcrypt, nunca texto plano.
- Cambiar credenciales demo antes de despliegue real.
- Usar HTTPS en despliegue real.
- Restringir CORS en producción para dominios permitidos.
- Evitar publicar dumps con información real de usuarios.
- Usar variables de entorno diferentes por ambiente.
- Documentar y versionar cambios de base de datos mediante migraciones.

18. Solución de problemas

Problema	Causa probable	Acción recomendada
MySQL no conecta	Credenciales o nombre de base incorrectos.	Revisar DB_HOST, DB_USER, DB_PASSWORD y DB_NAME en .env.
Backend no levanta	Puerto ocupado o dependencias no instaladas.	Verificar puerto 3000, ejecutar npm install y revisar logs.
/api/servicios no responde	Backend detenido o ruta incorrecta.	Confirmar npm start y abrir http://localhost:3000/api/servicios .
Login falla	Credenciales incorrectas, hash	Validar usuario admin, roles y password_hash

Problema	Causa probable	Acción recomendada
	inválido o rol no existente.	en MySQL.
Frontend no abre	Servidor estático no iniciado o puerto ocupado.	Ejecutar npm start en cleanhome-admin-frontend y revisar puerto 5173.
Android emulador no conecta	Uso de localhost en lugar de 10.0.2.2.	Configurar BASE_URL como http://10.0.2.2:3000/api/.
Android físico no conecta	IP incorrecta o firewall bloqueando.	Usar IP local de la PC y permitir tráfico al puerto 3000.
Catálogo vacío	Servicios inactivos o base sin datos semilla.	Validar servicios.activo y respuesta de GET /api/servicios.
Solicitud no se crea	Token faltante o campos obligatorios incompletos.	Enviar Authorization Bearer y validar id_servicio, fecha, hora y dirección.
Estado no cambia	Usuario sin rol Admin o estado inválido.	Usar token admin y estados permitidos.

19. Mantenimiento

- Actualizar este manual cuando se agreguen nuevos endpoints, campos de base de datos o flujos Android.
- Crear migraciones para cambios de base de datos en lugar de modificar directamente bases existentes.
- Probar backend, web y Android antes de hacer merge a la rama principal.
- Mantener .gitignore actualizado para evitar credenciales y artefactos generados.
- Usar commits pequeños y descriptivos.
- Mantener README.md y documentos por módulo alineados con el código fuente.
- Validar que las credenciales demo no se usen en ambientes productivos.

20. Glosario

Término	Definición
API	Interfaz HTTP que permite que web y Android consuman funcionalidades del backend.
Backend	Servidor Node.js/Express que contiene lógica de negocio y acceso a MySQL.
JWT	Token firmado usado para autenticar usuarios.
Rol	Nivel de permiso del usuario, por ejemplo Cliente o Admin.
MySQL	Motor de base de datos central del sistema.
SQLite	Base de datos local dentro del dispositivo Android.
Retrofit	Librería Android para consumir APIs REST.
Endpoint	Ruta HTTP expuesta por el backend.
Middleware	Función intermedia en Express para validar autenticación, roles o datos.

Término	Definición
CRUD	Operaciones de crear, leer, actualizar y eliminar o desactivar registros.
Sincronización	Proceso para mantener datos locales y remotos consistentes.
client_temp_id	Identificador local generado por Android para evitar duplicados al sincronizar.
.env	Archivo local con variables sensibles de configuración.
Admin	Usuario con permisos administrativos.
Cliente	Usuario que solicita servicios de limpieza.

21. Conclusiones

CleanHome queda estructurado como un sistema modular con separación clara entre API, frontend web, aplicación Android y persistencia. Esta separación facilita el mantenimiento, la validación por módulos y un posible despliegue futuro.

El backend centraliza reglas de negocio, autenticación, roles y acceso a MySQL. El panel web y la aplicación Android consumen la misma API, lo que permite que los datos creados desde un cliente se reflejen en los demás módulos.

La aplicación Android incorpora SQLite y SharedPreferences como soporte local para sesión, catálogo e historial. MySQL sigue siendo la fuente oficial de datos, mientras que SQLite apoya la experiencia móvil y escenarios de sincronización.

Para que un desarrollador levante el proyecto, el flujo recomendado es crear la base desde cero, configurar .env, levantar backend, levantar frontend web, configurar ApiConfig.kt y ejecutar Android desde Android Studio.

Antes de llevar el sistema a un entorno real, deben reemplazarse credenciales demo, habilitar HTTPS, restringir CORS, proteger secretos por ambiente y documentar cualquier cambio de API o base de datos.